

Сравнительный анализ помехоустойчивости кодов Рида–Соломона и ВСН

Лабораторная работа по теории кодирования · направления А (реализации) и Б
(режимы)

Вадим Денисович

Июнь 2026

Содержание

1. Введение	2
1.1 Цель работы	2
1.2 Краткий обзор кодов РС и ВСН	2
1.3 Два направления сравнения	5
2. Методология	5
2.1 Используемые инструменты	5
2.2 Модели каналов	6
2.3 Параметры эксперимента	7
2.4 Метрики и статистика	7
3. Результаты: Направление А (сравнение библиотек)	9
3.1 Гипотеза	9
3.2 Эксперимент	9
3.3 Статистическая проверка	10
3.4 Интерпретация	11
4. Результаты: Направление Б (сравнение режимов)	11
4.1 Гипотеза	11
4.2 Эксперимент	11
4.3 Статистическая проверка	12
4.4 Интерпретация	13
5. Заключение	14

1. Введение

1.1 Цель работы

Провести сравнительный анализ помехоустойчивости кодов Рида–Соломона (РС) и БЧХ (ВСН) с использованием готовых библиотек кодирования/декодирования по двум направлениям:

- (А) сравнение двух независимых реализаций одного и того же кода;
- (Б) сравнение режимов работы кодов — разных кодов с одинаковой исправляющей способностью и разных моделей каналов.

1.2 Краткий обзор кодов РС и ВСН

Коды БЧХ — циклические коды, исправляющие кратные ошибки. Двоичный ВСН-код длины $n = 2^m - 1$ с исправляющей способностью t задаётся порождающим многочленом $g(x) = \text{НОК}(M_1(x), M_3(x), \dots, M_{2t-1}(x))$, где $M_i(x)$ — минимальный многочлен элемента α^i поля $\text{GF}(2^m)$. Корнями $g(x)$ служат $\alpha, \alpha^2, \dots, \alpha^{2t}$, что гарантирует конструктивное расстояние $d \geq 2t + 1$.

Коды Рида–Соломона — не двоичные ВСН-коды, у которых алфавит символов совпадает с полем локаторов: символы — элементы $\text{GF}(q)$, длина $n = q - 1$, порождающий многочлен $g(x) = (x - \alpha)(x - \alpha^2) \dots (x - \alpha^{2t})$ степени $n - k = 2t$. РС — МДР-коды (достигают границы Синглтона $d = n - k + 1$) и исправляют любые $t = (n - k)/2$ **символьных** ошибок. Поскольку ошибка в символе «стоит» одинаково независимо от числа испорченных бит внутри него, РС особенно эффективны против пакетных ошибок.

Декодирование обоих классов идёт по единой схеме:

1. вычисление синдромов $S_i = r(\alpha^i)$, $i = 1, \dots, 2t$;
2. нахождение многочлена локаторов ошибок $\Lambda(x)$ — **алгоритм Берлекэмпа–Мессис (БМ)**;
3. поиск корней $\Lambda(x)$ перебором — **процедура Чиеня**: позиции ошибок j удовлетворяют $\Lambda(\alpha^{-j}) = 0$;
4. для не двоичных кодов — значения ошибок по **формуле Форни** $e_j = \Omega(X_j^{-1})/\Lambda'(X_j^{-1})$, где $\Omega(x) = S(x)\Lambda(x) \bmod x^{2t}$ (для двоичных кодов значение ошибки равно 1 — бит инвертируется).

Ниже оба алгоритма продемонстрированы письменно на примерах в $\text{GF}(2^3)$. Все вычисления продублированы скриптом `tools/verify_manual_examples.py` (независимая реализация арифметики поля и БМ без библиотеки `galois`); кодовые слова используются как валидационные векторы.

Поле $\text{GF}(2^3)$

Поле строится по примитивному многочлену $p(x) = x^3 + x + 1$; примитивный элемент α — корень $p(x)$, откуда $\alpha^3 = \alpha + 1$. Элементы (запись битами $b_2b_1b_0 \leftrightarrow b_2\alpha^2 + b_1\alpha + b_0$):

Степень	Многочлен	Биты	Целое
0	0	000	0
α^0	1	001	1
α^1	α	010	2
α^2	α^2	100	4
α^3	$\alpha + 1$	011	3
α^4	$\alpha^2 + \alpha$	110	6
α^5	$\alpha^2 + \alpha + 1$	111	7
α^6	$\alpha^2 + 1$	101	5

Сложение — поразрядный XOR ($\oplus$) битовых записей; умножение — сложение степеней по модулю 7 (так как $\alpha^7 = 1$). В поле характеристики 2 вычитание совпадает со сложением.

Пример 1. Код РС(7, 3), $t = 2$ над $\text{GF}(2^3)$

Параметры: $n = 7$, $k = 3$, $n - k = 4 = 2t$, $d = 5$, исправляет $t = 2$ символьные ошибки.

Порождающий многочлен $g(x) = (x + \alpha)(x + \alpha^2)(x + \alpha^3)(x + \alpha^4)$. Раскрываем попарно:

- $(x + \alpha)(x + \alpha^2) = x^2 + (\alpha \oplus \alpha^2)x + \alpha^3 = x^2 + \alpha^4x + \alpha^3$, ведь $\alpha \oplus \alpha^2 = 010 \oplus 100 = 110 = \alpha^4$;
- $(x + \alpha^3)(x + \alpha^4) = x^2 + (\alpha^3 \oplus \alpha^4)x + \alpha^7 = x^2 + \alpha^6x + 1$, ведь $\alpha^3 \oplus \alpha^4 = 011 \oplus 110 = 101 = \alpha^6$.

Перемножая, $(x^2 + \alpha^4x + \alpha^3)(x^2 + \alpha^6x + 1)$ даёт

$$g(x) = x^4 + \alpha^3x^3 + x^2 + \alpha x + \alpha^3.$$

(Например, коэффициент при x^3 : $\alpha^6 \oplus \alpha^4 = 101 \oplus 110 = 011 = \alpha^3$; при x : $\alpha^4 \oplus \alpha^9 = \alpha^4 \oplus \alpha^2 = 110 \oplus 100 = 010 = \alpha$.)

Кодирование (систематическое). Сообщение $m = (\alpha, 1, \alpha^3)$, то есть $m(x) = \alpha x^2 + x + \alpha^3$. Кодовое слово $c(x) = m(x)x^4 + r(x)$, где $r(x) = m(x)x^4 \bmod g(x)$. Деление $m(x)x^4 = \alpha x^6 + x^5 + \alpha^3 x^4$ на $g(x)$ «столбиком»:

Шаг	Член частного	Текущий остаток
1	αx^2	$\alpha^5 x^5 + x^4 + \alpha^2 x^3 + \alpha^4 x^2$
2	$\alpha^5 x$	$\alpha^3 x^4 + \alpha^3 x^3 + \alpha^3 x^2 + \alpha x$
3	α^3	$r(x) = \alpha^4 x^3 + \alpha^2 x + \alpha^6$

Итог:

$$c(x) = \alpha x^6 + x^5 + \alpha^3 x^4 + \alpha^4 x^3 + \alpha^2 x + \alpha^6, \quad c = (\alpha, 1, \alpha^3, \alpha^4, 0, \alpha^2, \alpha^6).$$

Проверка: $c(\alpha^i) = 0$ для $i = 1, \dots, 4$ — слово принадлежит коду (подтверждено; кодер `galois` даёт то же слово — *валидационный вектор №1*).

Канал. Вносим две ошибки $e(x) = \alpha^2 x^5 + \alpha x^2$. Принято $r = c \oplus e = (\alpha, \alpha^6, \alpha^3, \alpha^4, \alpha, \alpha^2, \alpha^6)$.

Декодирование. Синдромы $S_i = r(\alpha^i) = e(\alpha^i)$. Развёрнуто для S_1 :

$$S_1 = r(\alpha) = 1 \oplus \alpha^4 \oplus 1 \oplus 1 \oplus \alpha^6 = \alpha^4 \oplus \alpha^6 \oplus 1 = 110 \oplus 101 \oplus 001 = 010 = \alpha,$$

аналогично $S_2 = 0$, $S_3 = \alpha$, $S_4 = \alpha^4$. Итак $S = (\alpha, 0, \alpha, \alpha^4)$.

Алгоритм Берлекэмпа–Месси. Начало: $\Lambda(x) = 1$, $B(x) = 1$, $L = 0$. Дискрепанс $d = S_k \oplus \sum_{i=1}^L \lambda_i S_{k-i}$; при $d \neq 0$ коррекция $\Lambda(x) \leftarrow \Lambda(x) \oplus d x B(x)$.

k	S_k	дискрепанс d	$\Lambda(x)$	L	$B(x)$
1	α	α	$1 + \alpha x$	1	α^6
2	0	α^2	1	1	$\alpha^6 x$
3	α	α	$1 + x^2$	2	α^6
4	α^4	α^4	$1 + \alpha^3 x + x^2$	2	$\alpha^6 x$

$\Lambda(x) = 1 + \alpha^3 x + x^2, \quad \deg \Lambda = L = 2 \Rightarrow \text{две ошибки.}$

Процедура Чиеня — перебор $j = 0, \dots, 6$, корни $\Lambda(\alpha^{-j}) = 0$:

j	α^{-j}	$\Lambda(\alpha^{-j})$
2	α^5	$1 \oplus \alpha^3 \cdot \alpha^5 \oplus \alpha^{10} =$ $1 \oplus \alpha \oplus \alpha^3 = 0$
5	α^2	$1 \oplus \alpha^3 \cdot \alpha^2 \oplus \alpha^4 =$ $1 \oplus \alpha^5 \oplus \alpha^4 = 0$

(Остальные j дают ненулевое значение.) Локаторы $X_1 = \alpha^2$, $X_2 = \alpha^5$ — ошибки в позициях x^2 и x^5 . Проверка факторизации: $(1 + \alpha^2 x)(1 + \alpha^5 x) = 1 + \alpha^3 x + x^2$. ✓

Формула Форни. $\Omega(x) = S(x)\Lambda(x) \bmod x^4 = \alpha + \alpha^4x$; производная $\Lambda'(x) = \alpha^3$ (член x^2 в характеристике 2 исчезает). Тогда

$$e_2 = \frac{\Omega(\alpha^5)}{\Lambda'(\alpha^5)} = \frac{\alpha^4}{\alpha^3} = \alpha, \quad e_5 = \frac{\Omega(\alpha^2)}{\Lambda'(\alpha^2)} = \frac{\alpha^5}{\alpha^3} = \alpha^2.$$

Исправление. $\hat{c} = r \oplus e$ восстанавливает $c = (\alpha, 1, \alpha^3, \alpha^4, 0, \alpha^2, \alpha^6)$ и сообщение $m = (\alpha, 1, \alpha^3)$.
✓

Пример 2. Код ВСН(7, 4), $t = 1$ над GF(2)

Двоичный ВСН-код в узком смысле (корни начинаются с α^1) длины $n = 7$ (эквивалент кода Хэмминга). Порождающий многочлен — минимальный многочлен элемента α : $g(x) = x^3 + x + 1$, $n - k = 3$, $k = 4$. Корни $\alpha, \alpha^2, \alpha^4$ (среди них α, α^2 подряд $\Rightarrow d \geq 3$).

Кодирование. $m = (1, 0, 0, 1)$, $m(x) = x^3 + 1$. Деление $m(x)x^3 = x^6 + x^3$ на $g(x)$ даёт остаток $r(x) = x^2 + x$, откуда $c(x) = x^6 + x^3 + x^2 + x$, $c = (1, 0, 0, 1, 1, 1, 0)$. Проверка $c(\alpha) = 0$ ✓ (валидационный вектор №2).

Канал. Ошибка в позиции x^4 : $r = (1, 0, 1, 1, 1, 1, 0)$.

Синдромы. $S_1 = r(\alpha) = \alpha^6 \oplus \alpha^4 \oplus \alpha^3 \oplus \alpha^2 \oplus \alpha = 110 = \alpha^4$; $S_2 = S_1^2 = \alpha^8 = \alpha$ (в характеристике 2 верно $r(\alpha^2) = r(\alpha)^2$).

БМ. На $k = 1$: $d = \alpha^4 \Rightarrow \Lambda = 1 + \alpha^4x$, $L = 1$. На $k = 2$: $d = S_2 \oplus \lambda_1 S_1 = \alpha \oplus \alpha^4 \cdot \alpha^4 = \alpha \oplus \alpha = 0$ — без изменений. Итог $\Lambda(x) = 1 + \alpha^4x$.

Корень. $\Lambda(\alpha^{-j}) = 0 \Leftrightarrow \alpha^j = \alpha^4 \Leftrightarrow j = 4$. Код двоичный — инвертируем бит в позиции x^4 :

$$\hat{c} = (1, 0, 0, 1, 1, 1, 0) = c, \quad m = (1, 0, 0, 1). \checkmark$$

1.3 Два направления сравнения

- **Направление А** — реализация одного кода РС(255, 223, 16) над GF(2⁸) библиотеками `galois` и `reedsolo` в идентичных условиях (канал BSC, общие N_{blocks} , p , `seed`); измеряются FER, BER и время декодирования на блок.
- **Направление Б** — на одной библиотеке (`galois`, чтобы исключить фактор реализации) сравниваются коды РС(15, 11, 2) над GF(2⁴) и двоичный ВСН(15, 7, 2) (оба $n = 15$, $t = 2$) в каналах BSC и Burst (пакет до 8 бит).

2. Методология

2.1 Используемые инструменты

Библиотека	Версия	Назначение	Источник
<code>galois</code>	0.4.11	Кодеки РС и ВСН, арифметика $GF(2^m)$ поверх <code>numpy</code> (JIT через <code>numba</code>)	github
<code>reedsolo</code>	1.7.0	Кодек РС на чистом Python (настраиваемые <code>fcr</code> , <code>prim</code>)	github
<code>numpy</code>	2.4.6	Векторные операции, генератор PCG64	numpy.org
<code>matplotlib</code>	3.10.9	Графики	matplotlib.org
<code>pytest</code>	8.x	Тесты	pytest.org

Обоснование выбора. `galois` и `reedsolo` — две наиболее распространённые Python-реализации РС с принципиально разной архитектурой: `galois` строит типизированные GF-массивы и компилирует операции через `numba`, `reedsolo` — компактная референсная реализация на чистом Python. Обе используют примитивный многочлен 0x11D для $GF(2^8)$ и декодер Берлекэмп–Мессе, что делает их сравнимыми. `galois` дополнительно предоставляет ВСН-кодек, поэтому направление Б проводится внутри одной библиотеки. Модели каналов, цикл Монте-Карло, метрики и статистика реализованы самостоятельно ([src/codingtheory/](#)).

Согласование реализаций. По умолчанию `galois.ReedSolomon` использует первый корень α^1 (в узком смысле, $c = 1$), а `reedsolo` — α^0 ($fcr = 0$). Для побайтовой идентичности кодовых слов в `reedsolo` задано $fcr = 1$; идентичность проверена валидационным вектором №4.

2.2 Модели каналов

Все каналы реализованы самостоятельно ([channels.py](#)), работают с кадрами битов и принимают внешний генератор случайных чисел (фиксированный `seed` $\Rightarrow$ воспроизводимость).

BSC (двоичный симметричный канал). Каждый бит независимо инвертируется с вероятностью p :

$$y_i = x_i \oplus e_i, \quad e_i \sim \text{Bernoulli}(p) \text{ i.i.d.}$$

для каждого бита `x_i` кадра:

```
если rand() < p: y_i = x_i XOR 1
иначе:          y_i = x_i
```

SymbolError (символьный канал). Кадр делится на m -битные символы; каждый символ с вероятностью p_s заменяется на случайный другой символ (XOR с равновероятной ненулевой маской из $GF(2^m) \setminus \{0\}$):

$$s_i \leftarrow s_i \oplus u_i, \quad u_i \sim \mathcal{U}(GF(2^m) \setminus \{0\}) \text{ с вер. } p_s.$$

для каждого символа `s_i` (m бит):

```
если rand() < p_s: s_i = s_i XOR randint(1 .. 2^m-1)
```

Burst (пакетный канал). С вероятностью p_{burst} на кадр вносится один пакет: непрерывный участок случайной длины $L \sim \mathcal{U}\{1, \dots, L_{\text{max}}\}$ со случайным началом $s \sim \mathcal{U}\{0, \dots, n_{\text{bits}} - L\}$; биты участка инвертируются.

```
если rand() < p_burst:
    L = randint(1 .. L_max); s = randint(0 .. n_bits - L)
    для i в [s, s+L): y_i = x_i XOR 1
```

Параметры: p (BSC), (p_s, m) (SymbolError), $(p_{\text{burst}}, L_{\text{max}}=8)$ (Burst). Валидация: эмпирические частоты ошибок согласуются с заданными в пределах 4σ , для Burst дополнительно проверены непрерывность пакета и ограничение длины (вектор №5, [tests/test_channels.py](#)).

2.3 Параметры эксперимента

Параметр	Направление А	Направление Б
Коды	PC(255, 223, 16), GF(2 ⁸)	PC(15, 11, 2), GF(2 ⁴) и BCH(15, 7, 2)
Реализации	galois vs reedsolo	galois
Канал	BSC	BSC и Burst ($L \leq 8$ бит)
N_{blocks} / точку	10 ⁴	10 ⁵
p -сетка	8 точек, [0.004; 0.015]	BSC: 10 точек [0.003; 0.3]; Burst: 10 точек [0.001; 0.3]
seed	20260 (общий)	20261 (общий)

Коды направления Б имеют одинаковую длину $n = 15$ и одинаковую исправляющую способность $t = 2$, то есть одинаковую избыточность $n - k = 4$ в единицах алфавита. В битах избыточности различаются: ВСН — 8 бит на 15-битный кадр (53.3%), РС — 4 символа = 16 бит на 60-битный кадр (26.7%); это различие обсуждается в §4.4.

$N_{\text{blocks}} = 10^5$ для направления Б обеспечивает значимость при $\text{FER} \sim 10^{-3}$ (~ 100 ошибочных кадров). Для направления А выбрано $N_{\text{blocks}} = 10^4$ (по критерию гипотезы А2, $N \geq 10^4$): декодер reedsolo на чистом Python сделал бы 10⁵ блоков многочасовыми без улучшения выводов — сравнение идёт в области «колена», где $\text{FER} \geq 10^{-2}$.

2.4 Метрики и статистика

FER (Frame Error Rate) — доля кадров с ошибкой декодирования; кадр ошибочен, если декодированное сообщение $\neq$ переданному. При отказе декодера применяется одинаковая для всех обёрток стратегия passthrough (берётся информационная часть принятого слова).

$$\text{FER} = \frac{N_{\text{err frames}}}{N_{\text{blocks}}}, \quad \text{BER} = \frac{N_{\text{err bits}}}{N_{\text{blocks}} \cdot k_{\text{bits}}}, \quad \text{Throughput} = \frac{N_{\text{blocks}} \cdot k_{\text{bits}}}{T_{\text{decode}}} \text{ [бит/с]}.$$

Время измеряется только для операции декодирования; JIT-компиляция galois выполняется до замеров (предварительный прогрев).

Доверительные интервалы (95%) — интервал Вальда для доли $\hat{p} = k/N$:

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1-\hat{p})}{N}}.$$

Проверка значимости — двухвыборочный z -тест для долей (pooled):

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\bar{p}(1-\bar{p})(1/N_1 + 1/N_2)}}, \quad \bar{p} = \frac{k_1 + k_2}{N_1 + N_2}, \quad \text{p-value} = \text{erfc}\left(\frac{|z|}{\sqrt{2}}\right);$$

различие значимо при $\text{p-value} < 0.05$.

Валидация на известных векторах ([validation.py](#), входит в `pytest`):

1. РС(7, 3): ручное кодовое слово (2, 1, 3, 6, 0, 4, 5) из §1.2 = кодер `galois`;
2. ВСН(7, 4): ручное слово (1, 0, 0, 1, 1, 1, 0) из §1.2 = кодер `galois`;
3. ВСН(15, 7): $g(x)$ из `galois` = табличный $g(x) = x^8 + x^7 + x^6 + x^4 + 1$ (Лин–Костелло, табл. 6.1);
4. РС(255, 223): `galois` и `reedsolo` ($fcr = 1$) дают побайтово идентичные слова;
5. статистическая валидация трёх каналов.

Дополнительно кривая FER Монте-Карло для РС(15, 11, 2) в символьном канале сопоставлена с точной теорией $\text{FER} = 1 - \sum_{i=0}^t \binom{15}{i} p_s^i (1 - p_s)^{15-i}$:

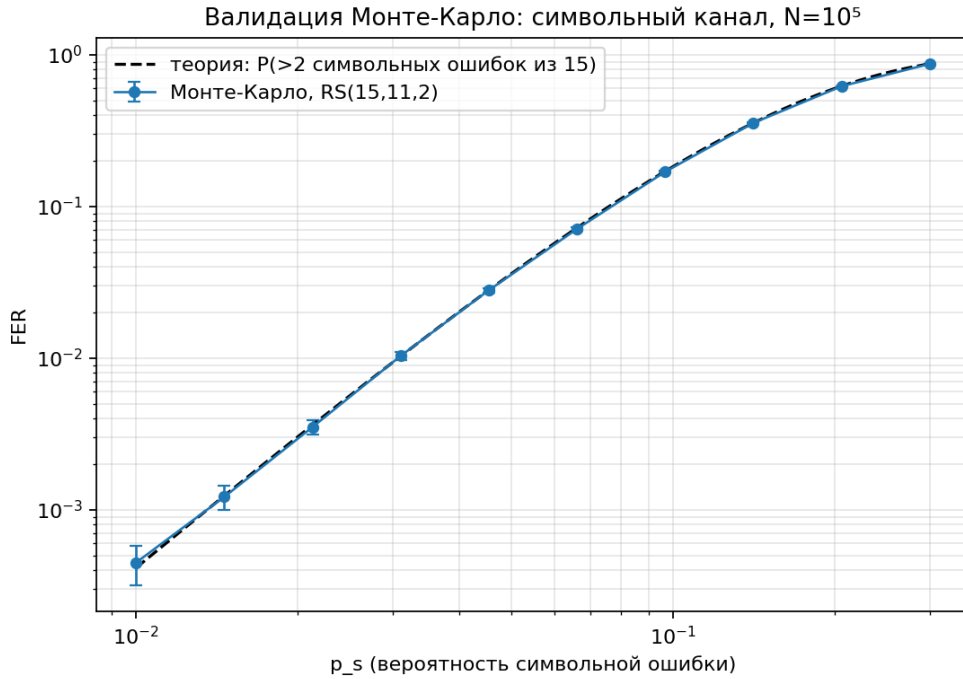


Рис. 1: Валидация Монте-Карло: символьный канал. Точки с 95%-ми доверительными интервалами совпадают с точной теоретической кривой на всём диапазоне.

Эмпирическая кривая совпала с теоретической, «колено» проходит около $p_s \approx t/n = 2/15 \approx 0.133$ — цикл Монте-Карло и символьный канал корректны.

3. Результаты: Направление А (сравнение библиотек)

3.1 Гипотеза

A1. `galois` и `reedsolo` дадут статистически неразличимые FER при декодировании РС(255,223) в канале BSC, так как обе реализуют стандартный алгоритм Берлекэмпа–Мессе.

A2. `galois` будет в 3–5 раз быстрее `reedsolo` благодаря векторизации через `numpy`, особенно при $N_{\text{blocks}} \geq 10^4$.

3.2 Эксперимент

Условия: код РС(255, 223, 16), канал BSC, $p \in [0.004; 0.015]$ (8 точек), $N_{\text{blocks}} = 10^4$, seed 20260 (общий); `reedsolo` с $fc = 1$ — кодовые слова идентичны `galois`.

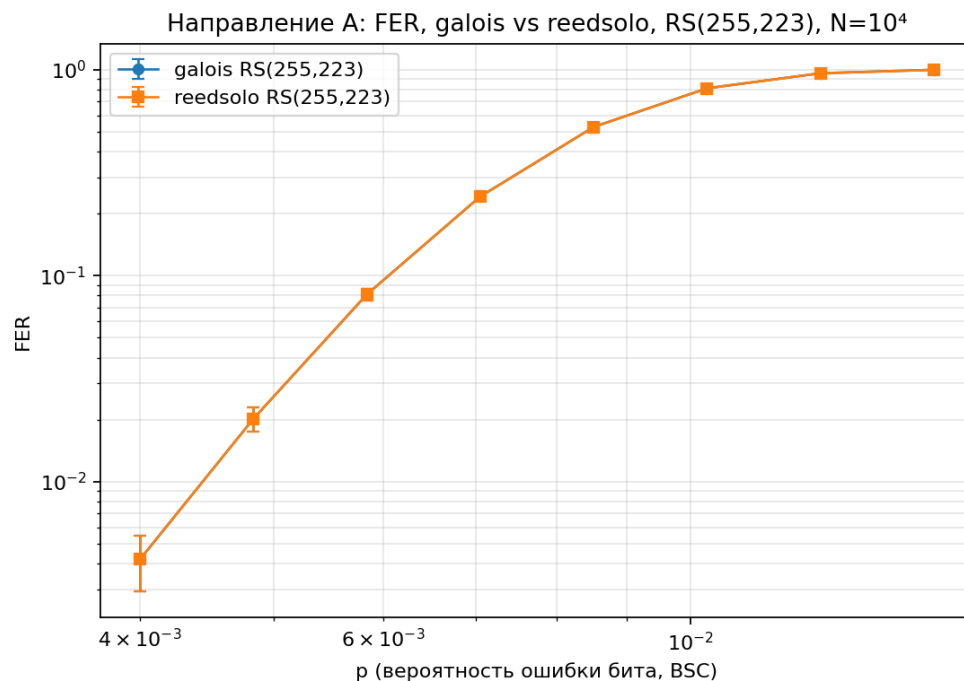


Рис. 2: Направление А: FER vs p . Кривые обеих библиотек полностью совпадают (маркеры наложены, доверительные интервалы перекрываются на 100%).

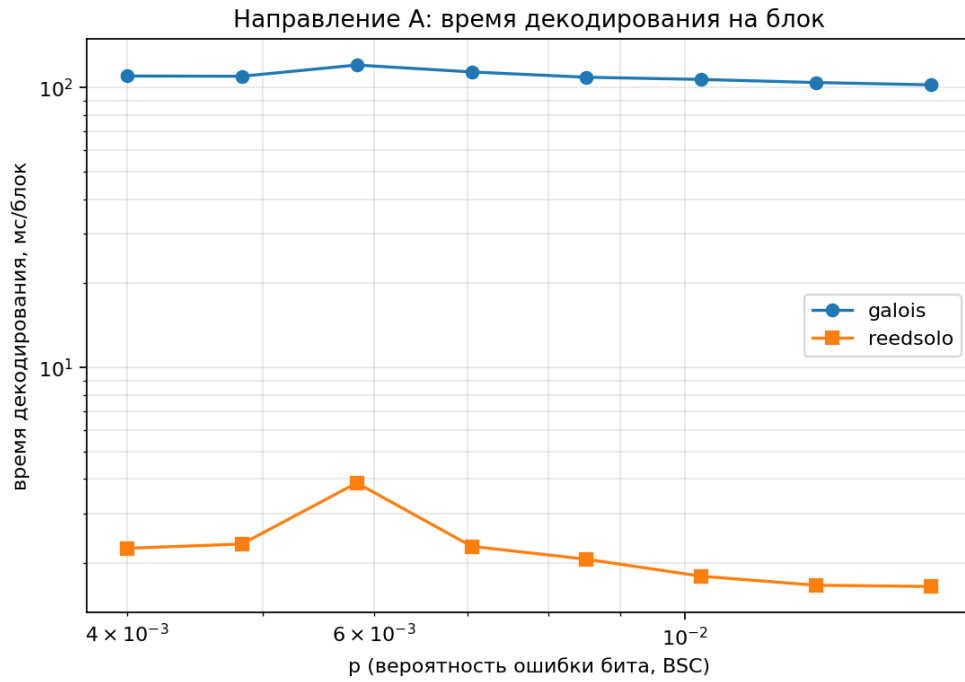


Рис. 3: Направление А: время декодирования на блок. **reedsolo** (нижняя кривая) на порядок быстрее **galois** (верхняя).

p	galois, мс/блок	reedsolo, мс/блок	galois/reedsolo	galois, кбит/с	reedsolo, кбит/с
0.0040	110.07	2.26	×48.7	16	789
0.0058	120.48	3.87	×31.1	15	461
0.0085	108.96	2.07	×52.7	16	863
0.0103	107.01	1.80	×59.5	17	992
0.0150	102.37	1.65	×62.0	17	1080

В среднем **galois** в ≈ 52 раза медленнее **reedsolo**. (Полные таблицы: [direction_a_timing.md](#), [direction_a_ztest.md](#).)

3.3 Статистическая проверка

p	FER galois	FER reedsolo	z	p-value	различие
0.0040	$4.20 \cdot 10^{-3}$	$4.20 \cdot 10^{-3}$	0.00	1.000	незначимо
0.0058	$8.10 \cdot 10^{-2}$	$8.10 \cdot 10^{-2}$	0.00	1.000	незначимо
0.0085	$5.259 \cdot 10^{-1}$	$5.259 \cdot 10^{-1}$	0.00	1.000	незначимо
0.0124	$9.587 \cdot 10^{-1}$	$9.587 \cdot 10^{-1}$	0.00	1.000	незначимо

На **всех** точках FER совпадает с точностью до одного ошибочного кадра: $z = 0$, p-value = 1.000. **Гипотеза A1 подтверждена** (в сильной форме).

3.4 Интерпретация

Почему FER совпадает точно. При общих seed , batch и (n, k) обе библиотеки получают побайтово идентичные сообщения, кодовые слова (выравнивание fcr) и реализации шума. Декодер с ограниченным расстоянием детерминирован: кадр исправляется $\Leftrightarrow$ в нём $\leq t = 16$ символьных ошибок — свойство принятого слова, а не реализации. Множества ошибочных кадров совпадают поэлементно, поэтому FER/BER равны точно; z -тест лишь формализует это.

Почему galois в ≈ 52 раза медленнее — гипотеза A2 опровергнута. Векторизация numba ускоряет арифметику поля, но декодер РС вызывается **поблочно**: БМ, Чиень и Форни для каждого из 10^4 кадров — отдельный вызов с диспетчеризацией numba и аллокацией GF-массивов. Для длинного кода $n = 255$ фиксированные накладные расходы на блок (≈ 110 мс) доминируют. reedsolo — чистый Python, но это компактный однопроходный декодер одного блока без инфраструктурного overhead, и здесь он выигрывает на порядок. Векторизация galois раскрылась бы при батч-обработке многих коротких слов, но не в режиме «один длинный блок за вызов».

Ответы на вопросы направления А.

1. *Одинаковы ли FER/BER?* — Да, идентичны с точностью до кадра ($z = 0$, $p = 1.0$): один код, один алгоритм, одинаковый вход.
2. *Насколько отличается время и почему?* — reedsolo в ≈ 52 раза быстрее на РС(255, 223); причина — накладные расходы galois на блок при длинном коде, где векторизация не реализуется.
3. *Что удобнее?* — reedsolo: простой байтовый API, мгновенный импорт, не требует numba. galois требовательнее (numba/llvmlite, JIT при первом вызове), но строго типизирован и единообразно работает с полем.
4. *Ограничения?* — reedsolo ориентирована на GF(2^8) и только РС; galois поддерживает произвольные GF(p^m), а также ВСН/Хэмминга/линейные коды — что и использовано в направлении Б.

4. Результаты: Направление Б (сравнение режимов)

4.1 Гипотеза

Б1. При пакетных ошибках длины ≤ 8 бит код РС(15, 11, 2) покажет FER в 3–5 раз ниже, чем ВСН(15, 7, 2) при той же избыточности, так как пакет затрагивает ≤ 2 –3 символа в GF(2^4), но до 8 битовых ошибок для двоичного кода.

Б3. В BSC коды РС и ВСН с одинаковой избыточностью покажут сопоставимый FER, но при переходе к пакетным ошибкам преимущество перейдёт к РС.

4.2 Эксперимент

Одна библиотека galois, коды РС(15, 11, 2) и ВСН(15, 7, 2) (оба $n = 15$, $t = 2$), $N_{\text{blocks}} = 10^5$, seed 20261, два канала.

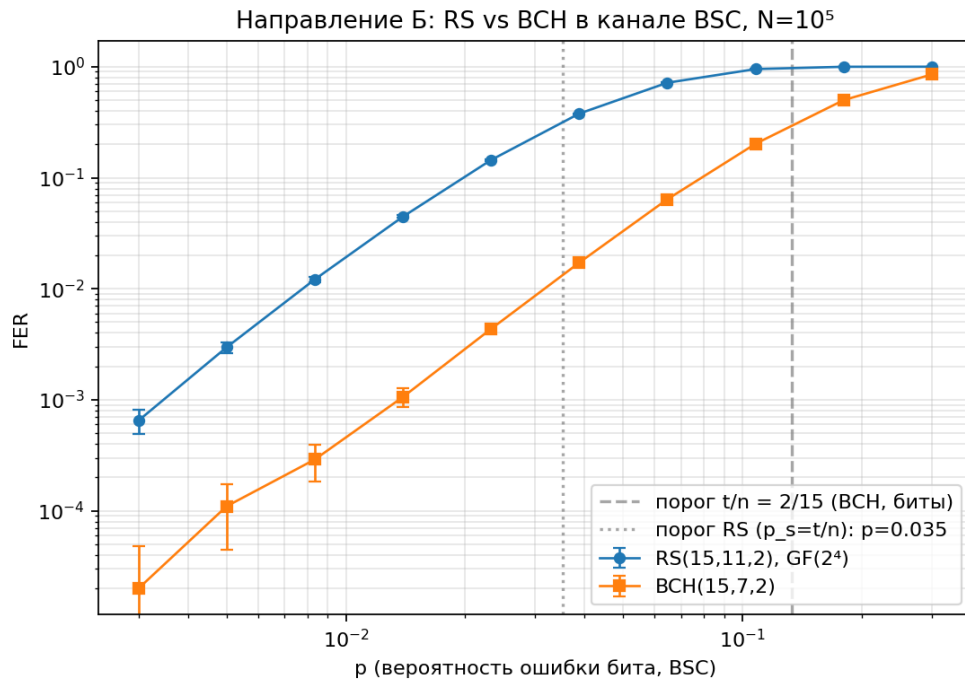


Рис. 4: Направление Б, канал BSC: FER vs p . BCH (нижняя кривая) на 1–2 порядка лучше РС. Линии — теоретические пороги $p \approx t/n$.

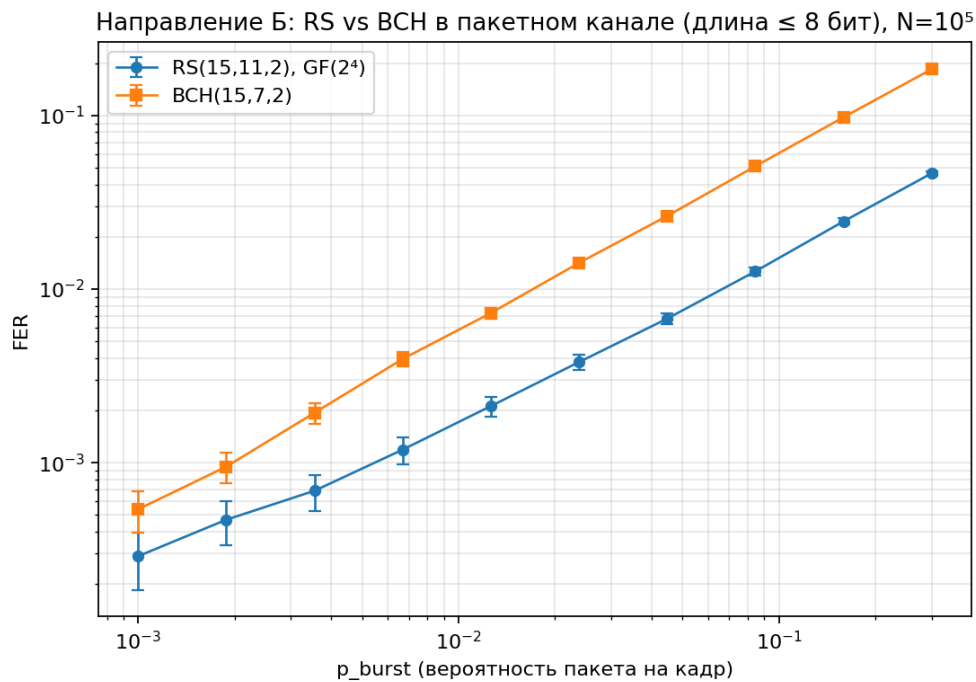


Рис. 5: Направление Б, пакетный канал ($L \leq 8$ бит): РС (нижняя кривая) в ≈ 3 –4 раза лучше BCH. Кроссовер относительно BSC.

4.3 Статистическая проверка

Канал BSC ($z > 0 \Rightarrow \text{FER РС выше}$): различие значимо ($p\text{-value} < 10^{-3}$) на всех точках, BCH строго лучше.

p	FER PC	FER BCH	z	p-value
0.0083	$1.214 \cdot 10^{-2}$	$2.90 \cdot 10^{-4}$	+33.7	< 0.001
0.0233	$1.445 \cdot 10^{-1}$	$4.35 \cdot 10^{-3}$	+119.4	< 0.001
0.1078	$9.519 \cdot 10^{-1}$	$2.021 \cdot 10^{-1}$	+339.3	< 0.001

Пакетный канал ($z < 0 \Rightarrow$ FER PC ниже): различие значимо на всех точках, PC строго лучше.

p_{burst}	FER PC	FER BCH	PC/BCH	z	p-value
0.0067	$1.19 \cdot 10^{-3}$	$3.96 \cdot 10^{-3}$	0.30	-12.2	< 0.001
0.0448	$6.76 \cdot 10^{-3}$	$2.636 \cdot 10^{-2}$	0.26	-34.3	< 0.001
0.3000	$4.642 \cdot 10^{-2}$	$1.846 \cdot 10^{-1}$	0.25	-96.7	< 0.001

Отношение $\text{FER}_{\text{PC}}/\text{FER}_{\text{BCH}} \approx 0.25\text{--}0.34$, то есть **PC даёт FER в $\approx 3\text{--}4$ раза ниже** — в диапазоне гипотезы Б1.

4.4 Интерпретация

«Одинаковая избыточность» в единицах алфавита различается в битах, а единицы исправления у кодов разные: PC считает **символьные** (4-битные) ошибки, BCH — **битовые**.

- **В BSC выигрывает BCH.** Независимые битовые ошибки рассеяны: символ $\text{GF}(2^4)$ повреждается с вероятностью $\approx 1 - (1 - p)^4 \approx 4p$. PC «развёрнут» в 60 бит и при бюджете $t = 2$ символа имеет вчетверо большую экспозицию, тогда как BCH защищает короткий 15-битный кадр вдвое большей долей проверочных бит. Итог — FER BCH на 1–2 порядка ниже. Это уточняет **Б3**: в BSC коды не сопоставимы.
- **В пакетном канале выигрывает PC.** Пакет ≤ 8 бит затрагивает $\leq \lceil 8/4 \rceil + 1 = 3$, чаще ≤ 2 соседних символа — в пределах $t = 2$ символа PC. Для двоичного BCH тот же пакет — до 8 битовых ошибок, вчетверо больше бюджета $t = 2$. Поэтому FER PC в $\approx 3\text{--}4$ раза ниже. **Гипотеза Б1 подтверждена.**

Как тип канала влияет на выбор кода. Принципиально: кроссовер кривых (BCH ниже в BSC, PC ниже в Burst) — прямая иллюстрация. Для независимых ошибок предпочтительны двоичные коды с высокой битовой избыточностью; для пакетных — символьные коды (PC), где пакет «стягивается» в малое число символов. Поэтому PC применяют против пакетных искажений (CD/DVD, QR, DVB), часто с перемежением. **Часть Б3 о переходе преимущества к PC подтверждена.**

Согласие с порогом $p \approx t/n$. «Колено» FER ожидается там, где среднее число ошибок на кадр достигает t :

- BCH в BSC: порог $p \approx t/n = 2/15 \approx 0.133$; крутой рост FER приходится на эту окрестность (FER ≈ 0.20 при $p \approx 0.108$, FER ≈ 0.50 при $p \approx 0.18$).
- PC в BSC: порог по символам $p_s = t/n$, в битах $p = 1 - (1 - 2/15)^{1/4} \approx 0.035$; кривая PC даёт FER ≈ 0.4 уже при $p \approx 0.039$.
- PC в символьном канале (§2.4): «колено» при $p_s \approx 0.133$, эмпирика совпала с теорией — прямое подтверждение порога.

5. Заключение

1. **Инструменты реализованы и провалидированы:** три модели каналов (BSC, SymbolError, Burst), цикл Монте-Карло, метрики BER/FER/Throughput с 95%-ми доверительными интервалами и z -тестом. Корректность подтверждена 5 известными векторами (включая ручные примеры §1 и побайтовое совпадение `galois` $\leftrightarrow$ `reedsolo`) и совпадением кривой Монте-Карло с точной теорией.
2. **Направление А.** Две реализации РС(255, 223) дают **идентичные** FER/BER ($z = 0$, $p\text{-value} = 1.0$) — гипотеза А1 подтверждена. По скорости `reedsolo` в ≈ 52 раза **быстрее** `galois` при поблочном декодировании — гипотеза А2 (об ускорении `galois` в 3–5 раз) **опровергнута**; причина — накладные расходы `galois` на вызов при гранулярности «один блок».
3. **Направление Б.** При равной символьной избыточности помехоустойчивость определяется структурой ошибок канала: в BSC лучше **ВСН**(15, 7) (на 1–2 порядка по FER), в пакетном канале лучше **РС**(15, 11) (FER в ≈ 3 –4 раза ниже). Гипотеза Б1 подтверждена, Б3 подтверждена в части пакетного канала и уточнена в части BSC. Положение «колена» согласуется с порогом $p \approx t/n$.

Главный вывод. Выбор между РС и ВСН при фиксированной избыточности диктуется не «силой» кода вообще, а соответствием единицы исправления (символ против бита) статистике ошибок канала. Выбор библиотеки — компромисс между удобством/скоростью (`reedsolo` для РС над $\text{GF}(2^8)$) и общностью (`galois` для произвольных полей и ВСН).

Исходный код, тесты и сырые данные: github.com/VadimDenisovich/fundamentals-of-cybernetics, каталог `coding-theory/`. Воспроизведение: `pytest`, затем `python experiments/run_direction_a.py` и `run_direction_b.py`, графики — `python experiments/plot_results.py`. Сборка PDF — `bash report/build.sh`.